Global Event Transformer (GET) for Scalable Temporal Link Prediction

Anonymous Author(s)

Abstract

Temporal link prediction on dynamic graphs is often evaluated by ranking a large set of candidate destinations for each query edge. Many temporal graph models build candidate-specific temporal neighborhoods or run message passing for each (u, v, t) pair, which can lead to substantial repeated computation under full-ranking evaluation. We introduce GET (Global Event Transformer), a sequence-based model that encodes a chronological window of interaction events once and reuses the resulting query representation to score candidates in parallel. GET tokenizes each event into a short sequence, optionally augments tokens with lightweight structure-aware node embeddings (e.g., a time-decayed GNN encoder or a TGN-style memory), and applies a decoder-only causal Transformer with continuous-time ALiBi to capture temporal dependencies. This separation between history encoding and candidate scoring enables efficient batched inference. Across five large-scale benchmarks from the Temporal Graph Benchmark (TGB) suite and six additional datasets, GET attains competitive accuracy while improving inference throughput by up to 21× over strong temporal graph baselines.

CCS Concepts

• **Computing methodologies** → **Machine learning**; • **Information systems** → **Data mining**; • **Networks** → *Network dynamics*.

Keywords

Temporal graphs, dynamic link prediction, event sequence modeling, Transformer, scalable inference

ACM Reference Format:

Anonymous Author(s). 2026. Global Event Transformer (GET) for Scalable Temporal Link Prediction. In *Proceedings of ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '26)*. ACM, New York, NY, USA, 18 pages.

1 Introduction

Real-world systems such as social platforms, financial networks, and recommender services generate massive streams of time-stamped interactions [10, 13]. These data are naturally represented as *temporal graphs*, where edges correspond to interaction events with continuous timestamps. A central task is *temporal link prediction*: given a source node and the interaction history up to time t, rank candidate destination nodes that are likely to interact next [8].

Where current methods struggle. Popular temporal graph models either (i) maintain and update per-node memories [24, 26] or (ii) construct query-specific neighborhoods and aggregate them with time-aware message passing [30, 33, 38]. These designs are effective, but they become increasingly costly under candidate-heavy evaluation protocols such as TGB, where each query requires scoring a large candidate set. In practice, substantial computation is repeated across candidates, and long-range signals still have to be routed through local neighborhoods or repeatedly refreshed node states.

Our approach. This paper introduces GET (Global Event Transformer), which casts temporal link prediction as *global event sequence modeling*. Instead of building a separate computation graph for every candidate edge, GET serializes a sliding window of recent interaction events into a single token sequence, models it with a decoder-only causal Transformer, and obtains a contextual query representation that is computed once per query. To handle continuous time, we equip attention with a continuous-time ALiBi bias, and we optionally inject lightweight structure-aware node embeddings via a time-decayed GNN encoder and/or a TGN-style memory module. Candidate destinations are then scored in a single batched operation using precomputed (or cached) candidate embeddings, following an "encode once, score many" pattern.

Contributions. We propose GET, a decoder-only Transformer that models temporal link prediction by attending over a *global event window* rather than building candidate-specific computation graphs. The model cleanly separates history encoding from candidate scoring, which makes the "encode once, score many" pattern natural under candidate-set evaluation. Across five large-scale TGB benchmarks and six additional datasets, GET achieves competitive accuracy while improving inference throughput by up to 21×. We also study how the context window length affects both accuracy and runtime, making the trade-off behind "global" attention explicit.

2 Related Work

Dynamic graph modeling [5, 7, 29, 36] has evolved along three major paradigms: memory-based methods, structure-enhanced methods, and Transformer-based global sequence models.

Memory-Based Methods. Memory-based methods maintain per-node states to model long-term temporal dependencies [3, 12, 44]. Early works like JODIE [15] and DyRep [26] update node embeddings through recurrent networks or temporal point processes. TGN [24] introduces a message-passing memory mechanism, establishing a standard template for many later models. Lightweight memory variants such as EdgeBank [22] replace parameterized updates with edge-level caching for scalability. Despite their strengths in capturing node histories, these models typically rely on local neighbor updates, limiting their global context modeling.

Structure-Based Methods. Structure-enhanced methods focus on encoding the neighborhood structure of each node [18, 19, 41]. TGAT [33] integrates temporal encoding with graph attention to aggregate local neighbors. CAWN [30] captures temporal neighborhood dynamics through random walk patterns. DyGFormer [38] and NAT [20] extend this line by combining neighborhood co-occurrence or multi-hop dictionaries with Transformer layers. However, these methods still operate within localized receptive fields and often scale poorly with large candidate sets.

KDD '26, (submission)
2026.

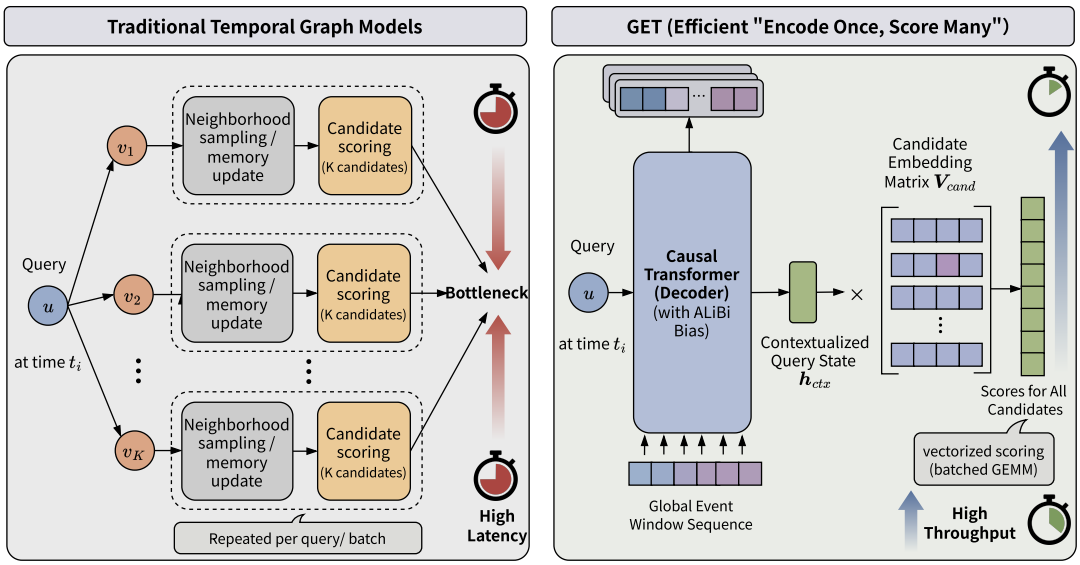


Figure 1: Why GET is faster at inference. Candidate-wise temporal graph pipelines repeatedly retrieve neighborhoods (or refresh node states) and score candidates for each query, which becomes a bottleneck when many destinations must be ranked (left). GET encodes a global event window once with a causal Transformer and scores all candidates via a single batched operation (right). Stopwatch and gradient annotations are illustrative, indicating the qualitative latency and throughput gap between the two workflows.

Sequence and Transformer-Based Methods. Inspired by advances in NLP, recent works propose sequence-centric modeling of dynamic graphs [1, 2, 31, 35]. For instance, SimpleDyG [32] treats the interactions of individual nodes as independent sequences. Recent sequence-based temporal models such as ROLAND [37] and DyGMamba [6] further explore state-space and edit-based architectures for dynamic interaction streams, but are mostly evaluated on dynamic graph benchmarks outside the TGB suite and often adopt task setups that are not directly aligned with our evaluation protocol. For reference, we include the recent sequence-based model DyGMamba in our unified small-scale benchmark comparison (Table 3); its original paper evaluates on different temporal graph benchmarks and task setups.

Key distinction. Sequence-based temporal models such as SimpleDyG and DyGFormer usually build a *node-centric* token sequence for each queried node. GET instead encodes a *global* stream of interaction events (within a sliding window), so any two events in the window can attend to each other directly. This global view complements node-centric modeling and is especially useful under candidate-set evaluation, where a single query must be scored against many destinations.

GET operates directly in continuous time (without snapshot discretization) and is evaluated under the TGB candidate-set temporal link prediction protocol.

3 Problem Formulation

A continuous-time dynamic graph (CTDG) is defined as $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is the set of nodes, and $\mathcal{E} = \{x_i = (s_i, d_i, t_i, \mathbf{m}_i)\}_{i=1}^{n}$ is the set of temporal interaction events. Here, $s_i, d_i \in \mathcal{V}$ are the source and destination nodes, $t_i \in \mathbb{R}^+$ is the timestamp, and $\mathbf{m}_i \in \mathbb{R}^{d_m}$ is an optional message feature. For modeling, we serialize the events in chronological order as $X_{1:n} = (x_1, x_2, \ldots, x_n)$, where $t_1 \leq t_2 \leq \cdots \leq t_n$. This serialization bridges the gap between graph-based dynamics and sequential modeling. Given historical observations $X_{1:n}$, one can model the next event $x_{n+1} = (s_{n+1}, d_{n+1}, t_{n+1})$ via the factorization

$$\begin{aligned} P(x_{n+1} \mid X_{1:n}) = P(s_{n+1} \mid X_{1:n}) \\ \cdot P(d_{n+1} \mid X_{1:n}, s_{n+1}) \\ \cdot P(t_{n+1} \mid X_{1:n}, s_{n+1}, d_{n+1}). \end{aligned} \tag{1}$$

This factorization provides a unified view that can support both link prediction and timestamp modeling by separating "who interacts with whom" from "when the interaction happens". In this paper we instantiate GET mainly as a discriminative model for predicting d_{n+1} (with an auxiliary regression loss for t_{n+1}), and use this factorization as a conceptual tool to organize the architecture.

4 Global Event Transformer (GET)

GET treats temporal link prediction as sequence modeling over a window of interaction events. Instead of building a separate computation graph for every candidate destination, it encodes a sliding event window once with a decoder-only causal Transformer and then scores all candidates in parallel. This "encode once, score many" separation is the main reason GET remains practical under candidate-heavy evaluation protocols such as TGB.

Throughout the paper, we use the term *global* in a windowed sense: self-attention is global *within the selected event window*, not over the entire unbounded history. We make this design choice explicit and study how the context window length impacts both

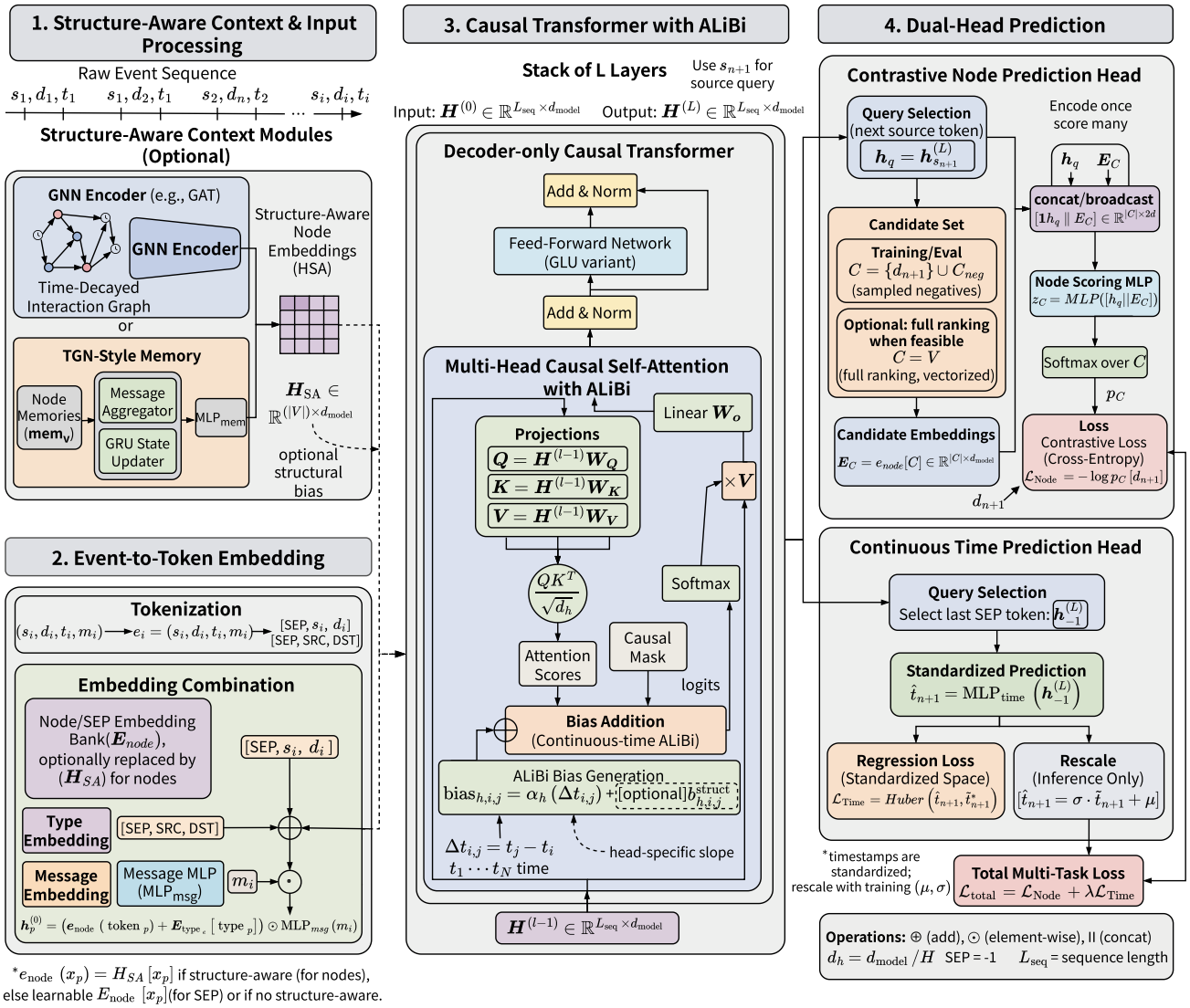


Figure 2: Overview of GET for dynamic link prediction. Each interaction event $e_i = (s_i, d_i, t_i, \mathrm{m}_i)$ is tokenized into a 3-token triplet $[\mathrm{SEP}, s_i, d_i]$ and embedded using (i) a learnable node/SEP lookup table, (ii) a type embedding for $\{\mathrm{SEP}, \mathrm{SRC}, \mathrm{DST}\}$, and (iii) a message projection. GET optionally injects structural priors by replacing node lookups with structure-aware embeddings H_{SA} produced by a lightweight GNN encoder or a TGN-style memory module. A decoder-only causal Transformer with continuous-time ALiBi biases encodes a global event window once, producing contextual token states $\mathrm{H}^{(L)}$. For node prediction, GET selects the source-token representation and scores a candidate set in parallel with an MLP over concatenated query–candidate embeddings ("encode once, score many"). For time prediction, GET regresses the next timestamp from the last SEP token and trains with a Huber loss in the standardized space, with rescaling applied at inference.

accuracy and runtime in Section 6.4. Figure 2 provides an overview of the model.

4.1 Event Window and Query Construction

GET does not assume access to an unbounded history. Instead, for each query time t_n we construct a *sliding window* of recent events and encode only this window with the Transformer. The window is capped by a token budget L_{seq}; since each event contributes three tokens, L_{seq} corresponds to roughly $L_{\text{seq}}/3$ past events.

We use the term "global" in a windowed sense: within the selected window, any two events can interact through self-attention in one step. This is substantially less local than neighborhood-based temporal GNNs, which route information through bounded-hop expansions, but it still keeps the quadratic attention cost under control.

For a link prediction query (s_{n+1}, t_{n+1}), we append a query prefix consisting of a separator token and the known source token s_{n+1}. The contextual state at this source position serves as the query representation used for scoring destination candidates (Section 4.4). In our implementation, L_{seq} is tuned on the validation split and therefore varies across datasets. The selected values are typically on the order of 10^3 tokens, and for some datasets (e.g., tgbl-wiki) we use a few thousand tokens. Section 6.4 quantifies how this choice trades off accuracy and throughput.

4.2 Event-to-Token Mapping Strategy

We serialize the event stream into a compact token sequence that a causal Transformer can process. Each interaction event $x_i = (s_i, d_i, t_i, \mathbf{m}_i)$ is represented by a fixed 3-token subsequence $[-1, s_i, d_i]$, where -1 denotes a special separator token (Fig. 2). This fixed-width design keeps the sequence short, makes event boundaries explicit, and keeps source/destination roles separately addressable; the query source token can therefore be reused directly as the candidate-scoring state. A window of n events yields $L_{\text{seq}} = 3n$ tokens.

Token embeddings combine three ingredients: (i) a node/SEP embedding lookup $\mathbf{H}_{SA}$, (ii) a token-type embedding $\mathbf{E}_{\text{type}}$ for {SEP, SRC,
DST}, and (iii) a message projection of the edge features $\mathbf{m}_i$. In the base model, $\mathbf{H}_{SA}$ is a learnable embedding table; Section 4.5 describes optional variants where $\mathbf{H}_{SA}$ is produced by a lightweight GNN encoder or a memory module. Formally, for token position p belonging to event i,

$$\mathbf{h}_p = \left(\mathbf{H}_{SA}[\text{token}_p] + \mathbf{E}_{\text{type}}[\text{type}_p]\right) \odot \text{MLP}_{\text{msg}}(\mathbf{m}_i), \tag{2}$$

where $\odot$ denotes element-wise multiplication.

We deliberately avoid introducing timestamps as separate tokens. A timestamp token would increase the sequence length from $3n$ to $4n$, which directly amplifies the $O(L_{\text{seq}}^2)$ self-attention cost on long histories. Instead, we handle continuous time through ALiBi-style biases (Section 4.3) and an auxiliary regression head (Section 4.4), which keeps a clean split between *who interacts* and *when*.

4.3 Global Dependency Modeling with ALiBi

The core of GET is a decoder-only Transformer with causal masking. To model continuous time without discretizing the stream, we add a continuous-time ALiBi bias to the attention logits [23]. Given the token sequence $\mathbf{H} \in \mathbb{R}^{L_{\text{seq}} \times d_{\text{model}}}$, we project embeddings to queries, keys, and values,

$$[\mathbf{Q}, \mathbf{K}, \mathbf{V}] = \mathbf{H} \cdot \mathbf{W}_{\text{QKV}}, \tag{3}$$

reshape them into H heads, and incorporate a head-specific bias,

$$\text{bias}_{h,i,j} = \alpha_h (t_j - t_i) + b^{\text{struct}}_{h,i,j}, \quad \alpha_h = 2^{-c(h+1)/H}, \tag{4}$$

where $t_j - t_i$ is the relative timestamp and α_h is a fixed slope (we use c=8). For each event triplet, all three tokens share the same timestamp t_i. Timestamps are standardized using the training-set mean and standard deviation. The optional $b^{\text{struct}}_{h,i,j}$ can encode graph priors, but the temporal term alone is typically sufficient.

The bias is added to the scaled dot product attention,

$$\text{Attn}(q_i, k_j, v_j) = \text{Softmax}\left(\frac{q_i \cdot k_j^\top}{\sqrt{d_h}} + \text{bias}_{h,i,j}\right) v_j, \tag{5}$$

which encourages temporally local focus while still allowing any token to attend globally within the window. Compared to sinusoidal or learned positional encodings, ALiBi is parameter-free and works well when sequence lengths and timestamp ranges vary. We adopt ALiBi by default; Appendix C.8 compares it with RoPE and learned absolute positions.

4.4 Decoder and Objectives

GET uses a two-head decoder: (i) contrastive node prediction for dynamic link prediction, and (ii) an auxiliary continuous-time regression head.

Contrastive Node Prediction. Given a partially observed event sequence $X_{1:n}$ and the next source node s_{n+1}, the model predicts the target node d_{n+1} via conditional contrastive classification. For each step, we construct a candidate set $C = \{d_{\text{true}}\} \cup \{d_{\text{neg},1}, \ldots, d_{\text{neg},K-1}\}$, where d_{true} is the ground-truth and $\{d_{\text{neg},i}\}$ are negatives from a pre-defined sampler. For each candidate $v_{\text{cand}} \in C$, the matching score is computed by concatenating the contextualized source representation $\mathbf{h}^{(L)}_{s_{n+1}}$ with the candidate embedding $\mathbf{h}^{\text{SA}}_{v_{\text{cand}}}$ (taken from $\mathbf{H}_{SA}$; Section 4.5), followed by an MLP, where $\|$ denotes vector concatenation:

$$\text{score}(s_{n+1}, v_{\text{cand}}) = \text{MLP}_{\text{node}}\left(\left[\mathbf{h}^{(L)}_{s_{n+1}} \| \mathbf{h}^{\text{SA}}_{v_{\text{cand}}}\right]\right), \tag{6}$$

Continuous Time Prediction. In parallel, GET predicts the timestamp t_{n+1} using the sequence-level representation $\mathbf{h}^{(L)}_{-1}$ of a special separator token:

$$\tilde{t}_{n+1} = \text{MLP}_{\text{time}}\left(\mathbf{h}^{(L)}_{-1}\right). \tag{7}$$

We standardize timestamps with the training mean μ and standard deviation σ and compute the regression loss in standardized space. At inference, we rescale to the original time unit via $\hat{t}_{n+1} = \sigma\, \tilde{t}_{n+1} + \mu$.

Training Objective and Optimization. GET is trained with a multi-task objective:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{node}} + \lambda \cdot \mathcal{L}_{\text{time}} \tag{8}$$

where λ balances the node classification (cross-entropy over C) and time regression terms. This combination encourages both accurate link prediction and robust modeling of temporal dynamics. Full details of the objective and optimization are provided in Appendix A. The two heads share the same contextual representation and are trained jointly; in our experiments this improves temporal calibration and reduces timestamp drift.

4.5 Optional Structure-Aware Context Modules

GET uses a node/SEP embedding table $\mathbf{H}_{SA} \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$ for both token embeddings (Section 4.2) and candidate representations (Section 4.4). In the simplest *base* configuration, $\mathbf{H}_{SA}$ is a standard learnable embedding matrix. When helpful, we optionally replace it with *structure-aware* embeddings produced by lightweight modules that inject local topological or temporal priors: a time-decayed GNN

encoder (*GNN-enhanced*) or a TGN-style memory module (*memory-enhanced*). These plug-ins affect only the embedding initialization; the Transformer over the event window remains the mechanism that models cross-event dependencies.

GNN-based Node Embeddings. To initialize nodes with structural priors, we compute structure-aware embeddings $\mathbf{H}_{GNN} \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$ by applying a single-pass GNN encoder on a **computational graph** constructed from recent events. For each edge (u, v, t_{uv}), we compute a k-dimensional time-decayed edge weight vector $\mathbf{w}_{uv,j}$ using exponential kernels:

$$\mathbf{w}_{uv,j} = \exp\left(-\alpha_j \cdot (t_{\text{current}} - t_{uv})\right), \quad \alpha_j > 0. \tag{9}$$

Equivalently, with $\Delta t_{uv} = t_{\text{current}} - t_{uv}$, the scalar time gap is mapped to the k-dimensional edge feature $\mathbf{w}_{uv} = [\exp(-\alpha_1 \Delta t_{uv}), \ldots, \exp(-\alpha_k \Delta t_{uv})]$. While exponential decay is a common heuristic [33], it may suppress signals from highly frequent interactions. Exploring more adaptive weighting schemes is a promising direction for future work.

We use Graph Attention Networks (GAT) [27] to incorporate both time-decayed weights and local neighborhood importance, and we found it to be robust across datasets. Appendix C.7 reports results with GCN and GraphSAGE variants, showing consistent trends. The encoder takes a learnable node embedding matrix $\mathbf{E}_{\text{init}}$ and computes:

$$\mathbf{H}_{GNN} = \text{GAT}(\mathbf{E}_{\text{init}}, \mathcal{G}_{\text{comp}}, \mathbf{w}_{uv,j}), \quad \mathbf{H}_{SA} \coloneqq \mathbf{H}_{GNN}. \tag{10}$$

Memory Module. To capture temporal dynamics, we employ a TGN-style memory module [24] that maintains a time-evolving hidden state $\mathbf{m}_v \in \mathbb{R}^{d_{\text{model}}}$ for each node. Whenever an event $(s_i, d_i, t_i, \mathbf{m}_i)$ occurs, the state is updated via a GRU using a message from the event context:

$$\begin{aligned} \mathbf{m}_v^{(t_i)} &= \text{GRU}\Big(\mathbf{m}_v^{(t_{i-1})},\ \text{MLP}_{\text{fuse}}(\mathbf{m}_u^{(t_{i-1})}, \mathbf{m}_i, \phi(t_i))\Big), \\ \mathbf{h}_v^{\text{SA}} &\coloneqq \text{MLP}_{\text{mem}}(\mathbf{m}_v). \end{aligned} \tag{11}$$

Here v is a participant node (e.g., s_i) and u the other (e.g., d_i), and MLP_{fuse} integrates the other node's memory, the event message $\mathbf{m}_i$, and the temporal encoding $\phi(t_i)$ of the timestamp.

5 Discussion: Expressiveness and Scalability

This section clarifies what the proposed formulation can represent and why it can be efficient in the candidate-set setting. Our intent is not to claim universal superiority of Transformers over temporal GNNs, but to make the trade-offs explicit.

Expressiveness relative to temporal message passing. Viewed on a fixed-length event window, GET combines two ingredients: (i) a tokenization that makes events and endpoints addressable by position, and (ii) causal self-attention that can directly connect distant events without repeated neighborhood expansion. The following results provide a simple formal perspective (proofs and precise definitions are deferred to Appendix B).

Proposition 1 (Approximation of temporal message passing). *On a fixed event window, a decoder-only Transformer with causal attention can approximate the computations of a k-hop temporal message passing network that aggregates information from the same window, up to arbitrary precision.*

Table 1: Per-query asymptotic cost under a candidate set C and an event window length L_{seq}. The Transformer term is quadratic in L_{seq} but independent of $|C|$; candidate scoring is linear in $|C|$ and can be fully vectorized.

Model family	History encoding	Candidate scoring
GNN/Mem (cand.)	$O(\lvert C\rvert \cdot \text{Enc})$	$O(\lvert C\rvert\, d)$
GET (ours)	$O(L_{\text{seq}}^2\, d_{\text{model}})$	$O(\lvert C\rvert\, d_{\text{model}})$

Proposition 2 (A separation from bounded-hop message passing). *There exist temporal link prediction instances where any bounded-hop temporal message passing model fails to recover the target dependency, while a causal Transformer can succeed by attending to a small number of informative events far apart in graph distance.*

Complexity and "encode once, score many". The practical bottleneck in many temporal graph pipelines is not the final scoring function, but the repeated construction of candidate-specific representations at query time. GET instead separates (i) history encoding over a global event window from (ii) scoring a candidate set, implemented as a single batched operation. Table 1 summarizes the dominant costs for one query under a sliding window of length L_{seq} and a candidate set C. Section 6.4 empirically studies how this window length trades off accuracy and runtime. Here, Enc denotes the cost of producing a time-conditioned embedding for one candidate node (often involving temporal neighbor retrieval and message passing or attention over a local subgraph). While some memory-based baselines can cache parts of this computation, full-ranking evaluation still requires touching every candidate state per query.

Limitations and scope. GET trades neighborhood-local computation for sequence modeling: attention scales as $O(L_{\text{seq}}^2)$ per layer, so very long histories require windowing or sparse attention. The optional structure-aware context modules partially mitigate this by injecting local structural priors without enlarging the Transformer window. Finally, our empirical study focuses on temporal link prediction (including full-ranking evaluation on TGB); extending the same event-sequence view to other supervision signals is an interesting direction, but is outside the scope of this paper.

6 Experiments

We evaluate GET on five large-scale TGB benchmarks and six commonly used dynamic-graph datasets. We report (i) main link-prediction performance, (ii) ablations on structural priors and the time auxiliary head, and (iii) inference throughput under the official TGB candidate-set protocol.

6.1 Experimental Setup

Datasets and Metrics. We evaluate GET on five large-scale TGB benchmarks [11], covering diverse dynamics ranging from long-range sparse interaction streams (e.g., tgbl-wiki, tgbl-comment) to high-volume short-horizon graphs (e.g., tgbl-flight), and six small-scale benchmarks focusing on behavioral dynamics [22]. Table 7 presents the statistical information of the TGB datasets. Following

Table 2: Link prediction performance (Test MRR) on large-scale TGB datasets.

Model	tgbl-wiki	tgbl-review	tgbl-coin	tgbl-comment	tgbl-flight
TGN	0.396 ±0.060	0.349 ±0.020	0.586 ±0.037	0.379 ±0.021	0.705 ±0.020
DyGFormer	**0.798 ±0.004**	0.224 ±0.015	0.752 ±0.004	0.670 ±0.001	NA
NAT	0.749 ±0.010	0.341 ±0.020	NA	NA	NA
CAWN	0.711 ±0.006	0.193 ±0.001	NA	NA	NA
GraphMixer	0.118 ±0.002	**0.521 ±0.015**	NA	NA	NA
TGAT	0.141 ±0.007	0.355 ±0.012	NA	NA	NA
TCL	0.207 ±0.025	0.193 ±0.009	NA	NA	NA
DyRep	0.050 ±0.017	0.220 ±0.030	0.452 ±0.046	0.289 ±0.033	0.556 ±0.014
EdgeBank (tw)	0.571	0.025	0.580	0.149	0.387
EdgeBank (un)	0.495	0.023	0.359	0.129	0.167
TNCN	0.718 ±0.001	0.377 ±0.010	**0.762 ±0.004**	0.697 ±0.006	0.820 ±0.004
GET	0.752 ±0.012	0.373 ±0.006	0.757 ±0.021	**0.732 ±0.017**	**0.895 ±0.003**

Table 3: Overall results for different baselines under the random negative sampling strategy.

Model	MOOC	LastFM	Enron	Can. Parl.	US Legis.	UN Trade
JODIE	80.04 ±1.77	74.83 ± 1.91	83.74 ± 0.35	64.09 ±0.37	66.86 ± 1.29	63.67 ± 0.29
DyRep	81.67 ± 0.36	76.01 ±1.55	80.97 ± 2.84	62.33 ±1.85	68.04 ±0.37	61.76 ±0.76
TGAT	85.73 ±0.15	75.33 ±0.21	69.93 ±0.88	65.46 ±0.53	61.42 ±2.21	61.16 ±0.14
TGN	89.12 ±1.25	78.68 ±2.94	84.77 ±0.84	65.25 ±1.59	69.11 ±0.37	63.45 ±1.24
CAWN	80.59 ±0.20	87.84 ±0.06	88.66 ±0.40	64.91 ±1.57	63.59 ±0.77	65.29 ±0.15
EdgeBank	65.73 ±0.11	77.22 ±0.96	81.72 ±0.46	60.91 ±0.38	56.03 ±0.56	60.87 ±0.08
TCL	82.06 ±0.18	72.48 ±1.45	78.68 ±0.51	64.12 ±1.82	62.06 ±0.45	62.20 ±0.18
GraphMixer	84.58 ±0.58	81.59 ±0.45	84.15 ±0.21	71.64 ±1.56	66.38 ±0.94	64.83 ±0.86
NAT	85.12 ±0.76	89.57 ±1.29	89.80 ±0.85	73.55 ±1.70	71.48 ±0.62	72.84 ±1.18
PINT	87.97 ±0.69	90.61 ±1.21	91.13 ±0.43	61.81 ±1.05	68.79 ±0.94	67.79 ±0.50
DyGFormer	87.32 ±0.36	93.45 ±0.09	91.73 ±0.26	93.99 ±0.53	64.63 ±1.82	65.86 ±0.85
DyGMamba	89.03 ±0.06	**93.71 ±0.15**	92.10 ±0.11	97.53 ±0.88	65.37 ±0.49	68.11±0.19
GET	**90.75 ±0.28**	88.39 ±1.35	**94.81 ±0.17**	**98.97 ±0.46**	**79.44 ±0.33**	**90.85 ±0.51**

standard practice, we report Mean Reciprocal Rank (MRR) for large-scale datasets and Average Precision (AP) for small-scale datasets, ensuring fair comparison with prior works.

Baselines and Implementation Details. We compare GET against a comprehensive suite of representative continuous-time temporal graph learning models, including: JODIE [15], DyRep [26], TGAT [33], TGN [24], CAWN [30], EdgeBank [22],TCL [28], GraphMixer [4], NAT [20],PINT [25], DyGFormer [38], TNCN [42], and the recent sequence-based model DyGMamba [6]. Baselines follow official or TGB-standard implementations; TGB scores are from the official leaderboard[1] to ensure consistency with prior works. GET is trained on a single RTX 4090 GPU. Per-dataset configs and all hyperparameters are in Appendix C.

Context windowing and evaluation step size. For each query, the Transformer encodes a sliding event window capped at L_{seq} tokens (Section 4.1). We tune L_{seq} per dataset on the validation split; the selected values are typically around 1024 tokens. At test time, we also set an evaluation step size (the number of queries processed per forward pass) to fit GPU memory. This step size influences throughput but does not change the computed ranking scores. For memory-based variants, we reset the memory state and warm it up by replaying the train+validation history before evaluating the test split, and we score each query before applying its update.

Negative sampling strategy. For evaluation on TGB, we follow the official protocol and use the released candidate sets and scripts [11], so every method is scored on exactly the same negatives. During training, we sample $K = 100$ random negative destinations per positive interaction, matching common practice in TGAT/TGN and the TGB examples. For the small-scale datasets in Table 3, we likewise use random negatives as described in the caption to keep the evaluation candidates aligned across methods.

Evaluation Protocol. All results are obtained by running the official implementations under the unified evaluation protocol. In all result tables, we highlight the best-performing model in **bold** and the second-best in underlined. "NA" indicates that an official leaderboard score was unavailable for that model–dataset pair; runtime failures are reported separately as OOM/OOT in the efficiency tables.

6.2 Main Results (RQ1: Overall Performance)

Table 2 summarizes test MRR on five large-scale TGB datasets. GET is particularly effective on the largest streams, reaching 0.895 on `tgbl-flight` and 0.732 on `tgbl-comment`. On `tgbl-wiki`, GET attains 0.752 MRR—below DyGFormer (0.798) but with substantially

[1] https://tgb.complexdatalab.com/

Table 4: Ablation results of GET on TGB datasets (Test MRR).

Model Variant	tgbl-wiki	tgbl-review	tgbl-coin	tgbl-comment	tgbl-flight
GET	0.439 ±0.009	0.293±0.017	0.608±0.009	0.365±0.013	0.848±0.011
GET w/o TimeLoss	0.408±0.005	0.276±0.011	0.599±0.010	0.337±0.014	0.842±0.025
GET+Memory	0.707±0.008	**0.373±0.006**	**0.757±0.021**	**0.732±0.017**	0.856±0.018
GET+Memory w/o TimeLoss	0.662 ±0.002	0.356±0.012	0.727±0.032	0.705±0.010	0.833±0.011
GET+GNN	**0.752±0.012**	0.309±0.025	0.648±0.014	0.615±0.008	**0.895±0.003**

higher candidate-set throughput (Section 6.5). On `tgbl-coin`, GET is close to the strongest neural baselines. On `tgbl-review`, GraphMixer leads the leaderboard; GET is comparable to TNCN/TGN and benefits most from the memory initializer. Overall, the preferred initializer is dataset dependent: the GNN variant is strongest on denser graphs (`tgbl-wiki`, `tgbl-flight`), while memory helps more in sparse or long-horizon regimes (`tgbl-review`, `tgbl-coin`, `tgbl-comment`).

We further evaluate on six widely used smaller benchmarks (Table 3) using average precision (AP) under a unified random-negative protocol. GET achieves the best AP on five datasets (MOOC, Enron, Can. Parl., US Legis., UN Trade). LastFM is more node-centric, with long per-user sequences; specialized sequence models (DyGMamba, DyGFormer) tend to be stronger, and GET remains competitive while prioritizing an event-window representation that is efficient under candidate-set ranking.

Table 5: Effect of the inference-time Transformer context window length L_{seq} on tgbl-wiki.

L_{seq} (tokens)	Test MRR	Throughput (q/s)
256	0.711 ±0.003	4976 ±87
512	0.722 ±0.003	4976 ±334
768	0.727 ±0.003	4896 ±358
1024	0.732 ±0.003	4941 ±169
1536	0.736 ±0.003	3885 ±33
2048	0.738 ±0.002	3781 ±70
3839	0.738 ±0.002	1330 ±8

6.3 Ablation Studies

We ablate the main design choices in GET on five large-scale TGB datasets. Starting from the base Transformer (learned node embeddings), we add the GNN-based initializer (+GNN) or the memory-based initializer (+Memory), and we also remove the auxiliary timestamp loss (w/o TimeLoss). Results are summarized in Table 4.

Tokenization trade-off. The triplet event serialization should be read as a representational/computational trade-off rather than as a universal winner. In the current audited **tgbl-wiki** control, fixed-event-budget results are 0.365 (`t1`), 0.349 (`t2`), and 0.356 (`t3`); fixed-token-budget results are 0.360, 0.349, and 0.356, respectively. These controls show that explicit event boundary and role markers can be useful, but the value of the extra tokens depends on the fairness criterion and the budget being held fixed.

Structure-aware initialization. The two structure-aware modules behave differently across regimes. On denser graphs such as **tgbl-wiki** and **tgbl-flight**, the GNN initializer provides the largest gains. On the extremely sparse **tgbl-review** stream, the memory variant is more effective. This suggests that no single structural prior dominates across all dynamic graphs.

Effect of auxiliary time prediction. Across both +GNN and +Memory, removing TimeLoss hurts the tuned main variants in Table 4, which motivates keeping the auxiliary timestamp head during training. A stricter placebo control clarifies the mechanism: on the current audited **tgbl-wiki** base-model rows, "none" and "true" are close (0.429 vs. 0.427), whereas shuffled-time supervision drops to 0.389 and a dummy scalar target collapses to 0.003. The appropriate interpretation is therefore not that the extra head universally improves every regime, but that arbitrary auxiliary targets are not interchangeable with real temporal supervision and that the magnitude of the benefit is dataset-dependent. We use $\lambda = 0.4$ throughout; Appendix C.9 shows that results are stable for $\lambda \in \{0.2, 0.4, 0.6\}$.

Temporal modeling capacity. Even without structural enhancements, the base Transformer remains competitive on **tgbl-flight** (MRR 0.848), indicating that much of GET's strength comes from sequence modeling. Section 6.4 further isolates the contribution of self-attention with inference-time attention controls.

6.4 Context Window Sensitivity and Measuring "Global" Context

GET applies global self-attention inside a sliding event window, so the token budget L_{seq} is an explicit modeling knob. Enlarging the window allows the model to access older events at the cost of higher quadratic attention overhead, while smaller windows improve throughput and memory usage.

Window length sweep. On tgbl-wiki, we train a single model with the default window ($L_{\text{seq}} = 3839$) and then truncate the available history at inference time. Unless noted otherwise, results are mean±std over three seeds. Because the checkpoint is fixed and only the inference-time context is varied, this analysis isolates how much historical context the trained Transformer actually uses, rather than replacing the best-tuned results in Table 2. Table 5 and Fig. 3 illustrate the accuracy–throughput trade-off as the inference-time window length varies. The same figure includes a control variant without the Transformer, whose performance stays around 0.49 MRR regardless of history length. Appendix Table 11 reports a complementary full-official sweep on tgbl-review for **GET+Memory**.

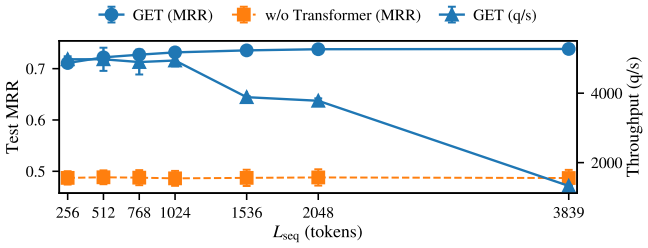


Figure 3: Test MRR and throughput as a function of the inference-time window length on tgbl-wiki.

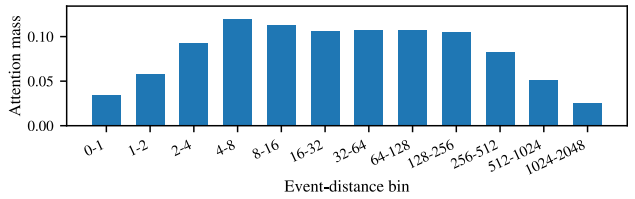


Figure 4: Attention distance profile on tgbl-wiki (event-distance bins; self-attention excluded).

In that sparse regime, enlarging the window from 256 to 2048 tokens increases MRR from 0.141 to 0.393, showing that longer event history can matter substantially when local structure is weak.

Attention controls. To disentangle the effect of global self-attention from the GNN initialization, we take the same trained checkpoint and modify the attention mask at test time. Disabling self-attention drops MRR from 0.738 to 0.473, and restricting attention to a local band of 256 tokens yields 0.721. Since the tokenization and input embeddings are unchanged, the gap is attributable to attention over the event window rather than to the structural encoder. Appendix Table 12 shows a complementary pattern on tgbl-review: local-band attention (span=256) is nearly identical to full attention (0.392 vs. 0.393), while disabling attention drops to 0.378. This suggests that in very sparse regimes much of the gain can come from retaining a sufficiently long event window, even when unrestricted all-to-all attention is less critical.

How global is the window? Window size alone does not indicate how far information can propagate structurally. For each query, we build the graph induced by events inside the window and compute the maximum shortest-path hop distance from the query source to any node that appears in the same window. On tgbl-wiki, the median and 90th percentile of this "max-hop-in-window" statistic are 1 and 8 hops; conditioning on query sources that appear in the window, the median is 2 and the 90th percentile is 10. This provides a concrete comparison to bounded-hop aggregation.

Figure 4 further characterizes how attention is distributed within the window. Although self-attention is causally masked, a non-trivial fraction of attention mass is assigned to distant events. On tgbl-wiki, roughly a quarter of the total attention lies beyond 128-event lags, suggesting that the model leverages information beyond immediate local context.

6.5 Efficiency and Scalability

We measure end-to-end evaluation time under the TGB protocol on four large-scale datasets, chosen to cover dense structure (tgbl-wiki), extreme sparsity (tgbl-review), a long temporal horizon (tgbl-coin), and massive edge volume (tgbl-flight).

Table 6: End-to-end inference time on an RTX 4090. GET+GNN is consistently faster than strong baselines under the official TGB protocol; DyGFormer exceeds memory/time limits on `tgbl-flight`.

Model	tgbl-wiki	tgbl-review	tgbl-coin	tgbl-flight
TGN	0:11:47	0:58:30	5:43:51	1:40:56
DyGFormer	1:26:39	4:50:15	7:46:44	OOM/OOT
GET+GNN	**0:00:33**	**0:02:59**	**0:25:28**	**1:03:39**

Measured operating-point cost makes the practicality story more concrete. In the current audited summary, **tgbl-wiki** GET+GNN trains at 594.2 seconds per epoch with 5526.7 MB peak GPU memory, while **tgbl-review** GET+GNN trains at 709.7 seconds per epoch with 2851.9 MB peak GPU memory. We report these as operating-point measurements rather than as a separate contribution claim. As shown in Table 6, **GET+GNN** achieves substantially lower end-to-end evaluation time across all datasets under the TGB protocol, and completes evaluation on **tgbl-flight** where DyGFormer runs out of memory or time. A primary bottleneck for many discriminative temporal graph models is the repeated work done *before* scoring candidates (e.g., neighborhood extraction, message passing, or memory updates). GET instead performs one self-attention pass over the current event window and then scores all candidates with a lightweight MLP. Candidate scoring is still $O(Kd)$ once node representations are available, so the gain we observe is operational rather than asymptotic.

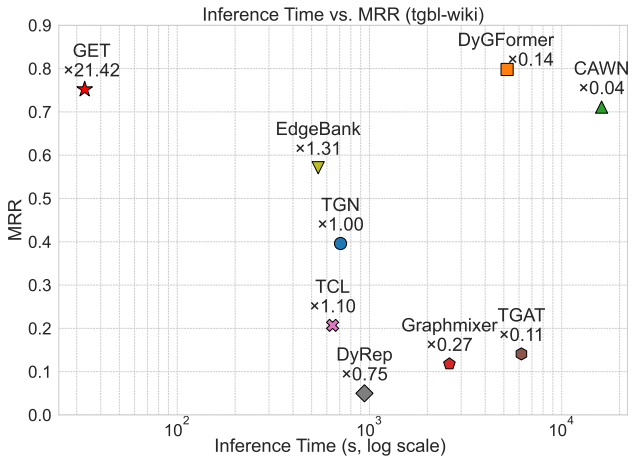


Figure 5: Scalability of GET on tgbl-wiki. Trade-off between inference time and MRR.

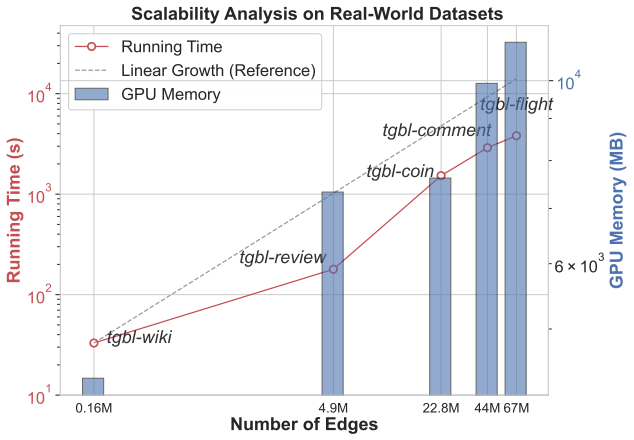


Figure 6: Scalability on TGB datasets, showing near-linear growth of inference time and GPU memory with dataset size.

Favorable trade-off and robust scalability. Fig. 5 illustrates this effect on tgbl-wiki: GET reaches high accuracy while being about 21× faster than TGN under the same evaluation protocol. Table 6 reports end-to-end evaluation time on four large-scale datasets; DyGFormer becomes impractical on tgbl-flight due to OOM/timeout, whereas GET remains practical across all scenarios on our hardware. Fig. 6 further shows that GET's runtime and peak GPU memory scale roughly linearly with edge count.

Limitations. Our scalability conclusions are drawn under the standardized TGB evaluation protocol, where inference is dominated by scoring large candidate sets on medium-length windows of recent events. In regimes with extremely long, unsegmented histories (very large L) or streaming settings without sliding windows, the quadratic self-attention term can become the dominant factor, and memory-based or linear-time architectures may be preferable. The audited context-composition controls also show that wider history is useful in some regimes but not all: for example, AP drops from 0.623 to 0.564 on **Can. Parl.**, from 0.514 to 0.465 on **UN Trade**, and from 0.661 to 0.640 on **US Legis.** when the history is restricted to source-only context, whereas **Enron** and **MOOC** are nearly source-local under the same control. These control values are measured on a fixed same-checkpoint evaluation slice under the small-dataset random-negative protocol, so the relevant signal is the within-row drop rather than direct comparison to the full Table 3 values. We therefore treat global event-window modeling as a regime-dependent operating point rather than as a universally superior bias.

7 Conclusion

In summary, we cast temporal link prediction on dynamic graphs as global event sequence modeling and introduced GET, a decoder-only causal Transformer equipped with continuous-time ALiBi and optional structure-aware context modules. By decoupling history encoding from candidate scoring, GET supports an "encode once, score many" inference pattern that is well matched to candidate-set evaluation. Across five large-scale TGB benchmarks and six additional datasets, GET achieves competitive accuracy while improving inference throughput by up to 21× over strong temporal graph baselines. We hope the event-sequence view complements neighborhood-local temporal graph models and motivates future work on longer-horizon event modeling and scalable long-context sequence architectures.

References

[1] Xi Chen, Yateng Tang, Jiarong Xu, Jiawei Zhang, Siwei Zhang, Sijia Peng, Xuehao Zheng, and Yun Xiong. 2025. Rethinking Time Encoding via Learnable Transformation Functions. In *Proceedings of the 40th International Conference on Machine Learning* (Vancouver, Canada) *(ICML'25)*. JMLR.org.

[2] Xi Chen, Yun Xiong, Siwei Zhang, Jiawei Zhang, Yao Zhang, Shiyang Zhou, Xixi Wu, Mingyang Zhang, Tengfei Liu, and Weiqiang Wang. 2024. DTFormer: A Transformer-Based Method for Discrete-Time Dynamic Graph Representation Learning. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management* (Boise, ID, USA) *(CIKM '24)*. Association for Computing Machinery, New York, NY, USA, 301–311. https://doi.org/10.1145/3627673.3679568

[3] Ke Cheng, Peng Linzhi, Junchen Ye, Leilei Sun, and Bowen Du. 2024. Co-Neighbor Encoding Schema: A Light-cost Structure Encoding Method for Dynamic Link Prediction. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Barcelona, Spain) *(KDD '24)*. Association for Computing Machinery, New York, NY, USA, 421–432. https://doi.org/10.1145/3637528.3671770

[4] Weilin Cong, Si Zhang, Jian Kang, Baichuan Yuan, Hao Wu, Xin Zhou, Hanghang Tong, and Mehrdad Mahdavi. 2023. Do We Really Need Complicated Model Architectures For Temporal Networks?. In *International Conference on Learning Representations*.

[5] Filip Cornell, Oleg Smirnov, Gabriela Zarzar Gandler, and Lele Cao. 2025. On the Power of Heuristics in Temporal Graphs. In *I Can't Believe It's Not Better: Challenges in Applied Deep Learning*. https://openreview.net/forum?id=cTckUeh0Sw

[6] Zifeng Ding, Yifeng Li, Yuan He, Antonio Norelli, Jingcheng Wu, Volker Tresp, Michael M. Bronstein, and Yunpu Ma. 2025. DyGMamba: Efficiently Modeling Long-Term Temporal Dependency on Continuous-Time Dynamic Graphs with State Space Models. *Transactions on Machine Learning Research* (2025). https://openreview.net/forum?id=sq5AJvVuha

[7] Julia Gastinger, Shenyang Huang, Mikhail Galkin, Erfan Loghmani, Ali Parviz, Farimah Poursafaei, Jacob Danovitch, Emanuele Rossi, Ioannis Koutis, Heiner Stuckenschmidt, Reihaneh Rabbany, and Guillaume Rabusseau. 2024. TGB 2.0: A Benchmark for Learning on Temporal Knowledge Graphs and Heterogeneous Graphs. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. https://openreview.net/forum?id=EADRzNJFn1

[8] Alessio Gravina and Davide Bacciu. 2024. Deep Learning for Dynamic Graphs: Models and Benchmarks. *IEEE Transactions on Neural Networks and Learning Systems* 35, 9 (2024), 11788–11801. https://doi.org/10.1109/TNNLS.2024.3379735

[9] Shubham Gupta, Sahil Manchanda, Srikanta Bedathur, and Sayan Ranu. 2022. TIGGER: Scalable Generative Modelling for Temporal Interaction Graphs. *Proceedings of the AAAI Conference on Artificial Intelligence* 36, 6 (Jun. 2022), 6819–6828. https://doi.org/10.1609/aaai.v36i6.20638

[10] Petter Holme and Jari Saramäki. 2012. Temporal networks. *Physics Reports* 519, 3 (2012), 97–125. https://doi.org/10.1016/j.physrep.2012.03.001 Temporal Networks.

[11] Shenyang Huang, Farimah Poursafaei, Jacob Danovitch, Matthias Fey, Weihua Hu, Emanuele Rossi, Jure Leskovec, Michael Bronstein, Guillaume Rabusseau, and Reihaneh Rabbany. 2023. Temporal Graph Benchmark for Machine Learning on Temporal Graphs. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 2056–2073.

[12] Shuo Ji, Mingzhe Liu, Leilei Sun, Chuanren Liu, and Tongyu Zhu. 2024. MemMap: An Adaptive and Latent Memory Structure for Dynamic Graph Learning. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Barcelona, Spain) *(KDD '24)*. Association for Computing Machinery, New York, NY, USA, 1257–1268. https://doi.org/10.1145/3637528.3672060

[13] Seyed Mehran Kazemi, Rishab Goel, Kshitij Jain, Ivan Kobyzev, Akshay Sethi, Peter Forsyth, and Pascal Poupart. 2020. Representation learning for dynamic graphs: a survey. *J. Mach. Learn. Res.* 21, 1, Article 70 (Jan. 2020), 73 pages.

[14] Srijan Kumar, Francesca Spezzano, VS Subrahmanian, and Christos Faloutsos. 2016. Edge weight prediction in weighted signed networks. In *Data Mining (ICDM), 2016 IEEE 16th International Conference on*. IEEE, 221–230.

[15] Srijan Kumar, Xikun Zhang, and Jure Leskovec. 2019. Predicting dynamic embedding trajectory in temporal interaction networks. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 1269–1278.

[16] Jérôme Kunegis. 2013. KONECT: The Koblenz Network Collection. In *Proceedings of the 22nd International Conference on World Wide Web* (Rio de Janeiro, Brazil)

(WWW '13 Companion). Association for Computing Machinery, New York, NY, USA, 1343–1350. https://doi.org/10.1145/2487788.2488173
[17] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. http://snap.stanford.edu/data.
[18] Xiaodong Lu, Leilei Sun, Tongyu Zhu, and Weifeng Lv. 2024. Improving Temporal Link Prediction via Temporal Walk Matrix Projection. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 141153–141182.
[19] Xiao Luo, Jingyang Yuan, Zijie Huang, Huiyu Jiang, Yifang Qin, Wei Ju, Ming Zhang, and Yizhou Sun. 2023. HOPE: High-order Graph ODE For Modeling Interacting Dynamics. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, 23124–23139. https://proceedings.mlr.press/v202/luo23f.html
[20] Yuhong Luo and Pan Li. 2022. Neighborhood-Aware Scalable Temporal Network Representation Learning. In *Proceedings of the First Learning on Graphs Conference (Proceedings of Machine Learning Research, Vol. 198)*, Bastian Rieck and Razvan Pascanu (Eds.). PMLR, 1:1–1:18.
[21] Christopher Morris, Martin Ritzert, Matthias Fey, William L. Hamilton, Jan Eric Lenssen, Gaurav Rattan, and Martin Grohe. 2019. Weisfeiler and Leman Go Neural: Higher-Order Graph Neural Networks. *Proceedings of the AAAI Conference on Artificial Intelligence* 33, 01 (Jul. 2019), 4602–4609. https://doi.org/10.1609/aaai.v33i01.33014602
[22] Farimah Poursafaei, Andy Huang, Kellin Pelrine, and Reihaneh Rabbany. 2022. Towards Better Evaluation for Dynamic Link Prediction. In *Thirty-sixth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
[23] Ofir Press, Noah Smith, and Mike Lewis. 2022. Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation. In *International Conference on Learning Representations*.
[24] Emanuele Rossi, Ben Chamberlain, Fabrizio Frasca, Davide Eynard, Federico Monti, and Michael Bronstein. 2020. Temporal Graph Networks for Deep Learning on Dynamic Graphs. In *ICML 2020 Workshop on Graph Representation Learning*.
[25] Amauri Souza, Diego Mesquita, Samuel Kaski, and Vikas Garg. 2022. Provably expressive temporal graph networks. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 32257–32269.
[26] Rakshit Trivedi, Mehrdad Farajtabar, Prasenjeet Biswal, and Hongyuan Zha. 2019. Dyrep: Learning representations over dynamic graphs. In *International conference on learning representations*.
[27] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. *International Conference on Learning Representations* (2018). https://openreview.net/forum?id=rJXMpikCZ accepted as poster.
[28] Lu Wang, Xiaofu Chang, Shuang Li, Yunfei Chu, Hui Li, Wei Zhang, Xiaofeng He, Le Song, Jingren Zhou, and Hongxia Yang. 2021. TCL: Transformer-based Dynamic Graph Modelling via Contrastive Learning. arXiv:2105.07944 [cs.LG]
[29] Ruijie Wang, Wanyu Zhao, Dachun Sun, Charith Mendis, and Tarek Abdelzaher. 2024. Towards Efficient Temporal Graph Learning: Algorithms, Frameworks, and Tools. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management* (Boise, ID, USA) *(CIKM '24)*. Association for Computing Machinery, New York, NY, USA, 5530–5533. https://doi.org/10.1145/3627673.3679104
[30] Yanbang Wang, Yen-Yu Chang, Yunyu Liu, Jure Leskovec, and Pan Li. 2021. Inductive Representation Learning in Temporal Networks via Causal Anonymous Walks. In *9th International Conference on Learning Representations*. OpenReview.net.
[31] Zhe Wang, Sheng Zhou, Jiawei Chen, Zhen Zhang, Binbin Hu, Yan Feng, Chun Chen, and Can Wang. 2025. Dynamic Graph Transformer with Correlated Spatial-Temporal Positional Encoding. In *Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining* (Hannover, Germany) *(WSDM '25)*. Association for Computing Machinery, New York, NY, USA, 60–69. https://doi.org/10.1145/3701551.3703489
[32] Yuxia Wu, Yuan Fang, and Lizi Liao. 2024. On the Feasibility of Simple Transformer for Dynamic Graph Modeling. In *Proceedings of the ACM Web Conference 2024* (Singapore, Singapore) *(WWW '24)*. Association for Computing Machinery, New York, NY, USA, 870–880. https://doi.org/10.1145/3589334.3645622
[33] Da Xu, Chuanwei Ruan, Evren Körpeoglu, Sushant Kumar, and Kannan Achan. 2020. Inductive representation learning on temporal graphs. In *8th International Conference on Learning Representations*. OpenReview.net.
[34] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How Powerful are Graph Neural Networks?. In *International Conference on Learning Representations*. https://openreview.net/forum?id=ryGs6iA5Km
[35] Lu Yi, Runlin Lei, Fengran Mo, Yanping Zheng, Zhewei Wei, and Yuhang Ye. 2025. Future Link Prediction Without Memory or Aggregation. arXiv:2505.19408 [cs.LG]
[36] Lu Yi, Jie Peng, Yanping Zheng, Fengran Mo, Zhewei Wei, Yuhang Ye, Yue Zixuan, and Zengfeng Huang. 2025. TGB-Seq Benchmark: Challenging Temporal GNNs with Complex Sequential Dynamics. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=8e2LirwiJT
[37] Jiaxuan You, Tianyu Du, and Jure Leskovec. 2022. ROLAND: Graph Learning Framework for Dynamic Graphs. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Washington DC, USA) *(KDD '22)*. Association for Computing Machinery, New York, NY, USA, 2358–2366. https://doi.org/10.1145/3534678.3539300
[38] Le Yu, Leilei Sun, Bowen Du, and Weifeng Lv. 2023. Towards better dynamic graph learning: New architecture and unified library. *Advances in Neural Information Processing Systems* 36 (2023), 67686–67700.
[39] Chulhee Yun, Srinadh Bhojanapalli, Ankit Singh Rawat, Sashank Reddi, and Sanjiv Kumar. 2020. Are Transformers universal approximators of sequence-to-sequence functions?. In *International Conference on Learning Representations*. https://openreview.net/forum?id=ByxRM0Ntvr
[40] Giselle Zeno, Timothy La Fond, and Jennifer Neville. 2021. DYMOND: DYnamic MOtif-NoDes Network Generative Model. In *Proceedings of the Web Conference 2021* (Ljubljana, Slovenia) *(WWW '21)*. Association for Computing Machinery, New York, NY, USA, 718–729. https://doi.org/10.1145/3442381.3450102
[41] Siwei Zhang, Xi Chen, Yun Xiong, Xixi Wu, Yao Zhang, Yongrui Fu, Yinglong Zhao, and Jiawei Zhang. 2024. Towards Adaptive Neighborhood for Advancing Temporal Interaction Graph Modeling. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Barcelona, Spain) *(KDD '24)*. Association for Computing Machinery, New York, NY, USA, 4290–4301. https://doi.org/10.1145/3637528.3671877
[42] Xiaohui Zhang, Yanbo Wang, Xiyuan Wang, and Muhan Zhang. 2024. Efficient Neural Common Neighbor for Temporal Graph Link Prediction. arXiv:2406.07926 [cs.LG]
[43] Dawei Zhou, Lecheng Zheng, Jiawei Han, and Jingrui He. 2020. A Data-Driven Graph Generative Model for Temporal Interaction Networks. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery; Data Mining* (Virtual Event, CA, USA) *(KDD '20)*. Association for Computing Machinery, New York, NY, USA, 401–411. https://doi.org/10.1145/3394486.3403082
[44] Hongkuan Zhou, Da Zheng, Xiang Song, George Karypis, and Viktor Prasanna. 2023. DistTGL: Distributed Memory-Based Temporal Graph Neural Network Training. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (Denver, CO, USA) *(SC '23)*. Association for Computing Machinery, New York, NY, USA, Article 39, 12 pages. https://doi.org/10.1145/3581784.3607056

A Training Algorithm Details

Training Objective and Optimization. The GET model is trained using a multi-task objective that jointly optimizes both node prediction and time prediction. The overall loss is formulated as a weighted sum of two components:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{node}} + \lambda \mathcal{L}_{\text{time}}, \tag{12}$$

where $\mathcal{L}_{\text{node}}$ denotes the node classification loss and $\mathcal{L}_{\text{time}}$ corresponds to the time regression loss, balanced by a hyperparameter λ.

For the node prediction task, we treat the predicted scores over the candidate set C as logits and apply the standard cross-entropy loss:

$$\mathcal{L}_{\text{node}} = -\log \frac{\exp(\text{score}(s_{n+1}, d_{\text{true}}))}{\sum_{v \in C} \exp(\text{score}(s_{n+1}, v))}. \tag{13}$$

This objective encourages the model to assign the highest probability to the true target node among all candidates.

For time prediction, we adopt the Huber loss:

$$\mathcal{L}_{\text{time}}(t, \hat{t}) = \begin{cases} \frac{1}{2}(t - \hat{t})^2, & \text{if } |t - \hat{t}| \le \delta, \\ \delta|t - \hat{t}| - \frac{1}{2}\delta^2, & \text{otherwise,} \end{cases} \tag{14}$$

where t is the ground-truth timestamp, $\hat{t}$ is the predicted timestamp, and δ is a threshold parameter, typically set to 1.0. Compared to standard L2 (mean squared error), the Huber loss behaves as L2 for small errors while transitioning to L1 (absolute error) for large

Algorithm 1 GET Training Procedure

Require: Training event set $\mathcal{E}_{\text{train}}$, model parameters θ, learning rate η, batch size B, loss weight λ
Ensure: Optimized parameters θ^*

```
for epoch = 1 to max_epochs do
    // Step 1: GNN-based structural encoding
    Construct historical graph G_hist from E_train
    Z_GNN ← GNN-Enhance(G_hist)
    for each batch B ⊂ E_train do
        // Step 2: Event serialization and token embedding
        X_token, Y_node, Y_time ← Tokenize(B, Z_GNN)
        C_nodes ← GetCandidates(B)
        // Step 3: Forward pass
        H_context ← Transformer(X_token)
        Score_node ← DecodeNode(H_context, C_nodes, Z_GNN)
        t̂_pred ← DecodeTime(H_context)
        // Step 4: Loss and parameter update
        L_node ← CrossEntropy(Score_node, Y_node)
        L_time ← HuberLoss(t̂_pred, Y_time)
        L ← L_node + λ · L_time
        θ ← AdamW(θ, ∇_θ L, η)
    end for
end for
θ* ← θ
return θ*
```

Algorithm 2 GET Inference Procedure (Autoregressive Generation)

Require: Trained model θ^*, historical sequence X_{hist}, number of future events k
Ensure: Generated future events X_{pred}

```
X_pred ← ∅
// Step 1: Encode structure once
Z_GNN ← GNN-Enhance(X_hist)
for i = 1 to k do
    // Step 2: Tokenize full history
    X_token ← Tokenize(X_hist, Z_GNN)
    H_context ← Transformer(X_token)
    // Step 3: Generate next event
    s_new ← PredictSource(H_context)
    C_nodes ← V // candidate set
    // avoid overlong line by splitting
    Score_node ←
        DecodeNode(H_context, s_new, C_nodes, Z_GNN)
    d_new ← arg max_{v∈C_nodes}(Score_node[v])
    t_new ← DecodeTime(H_context)
    x_new ← (s_new, d_new, t_new)
    X_hist ← X_hist ∪ {x_new}
    X_pred ← X_pred ∪ {x_new}
end for
return X_pred
```

deviations, making it more robust to noise and outliers. This robustness is particularly beneficial for real-world dynamic graphs where bursty and irregular events are common. The time prediction loss guides the model to better capture the rhythm and causal dependencies of events, which further enhances link prediction performance.

For optimization, we employ the AdamW optimizer with a cosine annealing learning rate scheduler combined with warm restarts. This combination promotes both stable convergence and improved training efficiency across diverse datasets.

For completeness and to ensure reproducibility, we provide the detailed pseudocode for the training procedure of GET in Algorithm 1 and the autoregressive inference procedure in Algorithm 2.

B Theoretical Details

B.1 Definitions

Definition 3 (Event sequence space)**.** Fix a maximum window length $L \in \mathbb{N}$ and feature dimension d. Let $\mathcal{X}_L$ denote the set of all event sequences $X = (x_1, \ldots, x_n)$ with $n \leq L$, where each event x_i has features in a bounded closed subset $\mathcal{B} \subset \mathbb{R}^d$. Under the product topology, $\mathcal{X}_L$ is a compact metric space.

Definition 4 (Function classes)**.** Let $\mathcal{F}_{\text{GNN},K}$ be the set of all functions $f : \mathcal{X}_L \to \mathbb{R}^m$ realizable by a fixed K-layer temporal MPNN architecture (message-passing GNN with K propagation steps). Let $\mathcal{F}_{\text{GET}}$ be the function class of GET models operating on the same input space $\mathcal{X}_L$.

Definition 5 (Uniform norm)**.** For $f, g : \mathcal{X}_L \to \mathbb{R}^m$, define

$$\|f - g\|_\infty \;:=\; \sup_{X \in \mathcal{X}_L} \|f(X) - g(X)\|_2.$$

Proposition 6 (Approximation). *For any $f \in \mathcal{F}_{\text{GNN},K}$ and any $\varepsilon > 0$, there exists a GET model $g \in \mathcal{F}_{\text{GET}}$ such that $\|f - g\|_\infty < \varepsilon$.*

Proposition 7 (Separation). *For any $K \geq 1$, there exist a prediction task h and event sequences $X, X' \in \mathcal{X}_L$ such that (i) $h(X) \neq h(X')$; (ii) any K-layer temporal MPNN produces identical outputs on X and X'; and (iii) a GET model can distinguish X and X'.*

B.2 Proof of Proposition 6

Proof. We argue in three steps. **Step 1: Continuity of temporal MPNNs.** A K-layer temporal MPNN on $\mathcal{X}_L$ is constructed by composing: (i) linear transformations on finite-dimensional feature vectors; (ii) pointwise nonlinearities such as ReLU or LeakyReLU; (iii) aggregations (e.g., sum/mean) over finite neighborhoods at each layer. Each of these operations is continuous, and finite compositions of continuous functions on a compact domain remain continuous. Thus any $f \in \mathcal{F}_{\text{GNN},K}$ belongs to $C(\mathcal{X}_L, \mathbb{R}^m)$.

Step 2: GET subsumes Transformers. Consider GET with: (i) the GNN encoder set to output a fixed learned embedding for each node, without any message passing; and (ii) the memory module disabled or set to an identity-like mapping that simply forwards its inputs. Under these constraints, the remaining part of GET is exactly a standard Transformer encoder with multi-head self-attention and feed-forward layers, equipped with positional/temporal encodings (ALiBi biases in our implementation, though sinusoidal encodings

would also suffice for the theoretical argument). Therefore $\mathcal{F}_{\text{GET}}$ contains the function class of Transformers with positional encodings as a subset.

Step 3: Universal approximation of Transformers. By the universal approximation result of Yun et al. [39], Transformers with positional encodings are universal approximators of continuous sequence-to-sequence functions on compact domains. In particular, for any $f \in C(\mathcal{X}_L, \mathbb{R}^m)$ and any $\varepsilon > 0$, there exists a Transformer T such that $\|f - T\|_\infty < \varepsilon$.

Conclusion. Combining Steps 1–3: any $f \in \mathcal{F}_{\text{GNN},K}$ is continuous (Step 1), and there exists a GET model that approximates f arbitrarily closely in the uniform norm (Steps 2–3). This proves Proposition 6. □

B.3 Proof of Proposition 7

Proof. We give an explicit construction.

Graph and sequences. Consider a path graph with nodes $v_0, v_1, \ldots, v_{K+1}$. Let $u := v_0$ and $v := v_{K+1}$, so that the shortest-path distance satisfies $\text{dist}(u, v) = K + 1$.

We construct two event sequences $X, X' \in \mathcal{X}_L$ as follows:

- Both sequences contain identical events (with identical features) involving nodes $v_1, \ldots, v_{K+1}$;
- X contains an additional event at node u with a binary feature $z = 1$;
- X' is the same as X except that this event at u has feature $z = 0$.

We define the prediction task as $h(X) := z$, i.e., the goal is to recover the binary feature associated with the distant node u.

MPNNs cannot distinguish X and X'. By the standard receptive-field property of K-layer MPNNs [21, 34], the representation $\mathbf{h}_v^{(K)}$ of node v after K message-passing layers depends only on nodes within K hops of v. Since $\text{dist}(u, v) = K + 1 > K$, node u lies outside this receptive field. Moreover, the K-hop neighborhoods of v in X and X' (including all features and events) are identical by construction. Therefore

$$\mathbf{h}_v^{(K)}(X) \;=\; \mathbf{h}_v^{(K)}(X'),$$

and any decoder based on $\mathbf{h}_v^{(K)}$ must produce identical outputs on X and X'. Hence no K-layer MPNN can implement h on this task.

GET can distinguish X and X'. In GET, each event (including the one at u) is encoded as a token in the global event sequence. The self-attention mechanism allows the token corresponding to the prediction at v to attend directly to the token for the event at u. By Proposition 6 and the universal approximation property of Transformers, there exists a parameterization of GET such that: (i) the query at v's position assigns high attention weight to u's event token, effectively extracting the feature z; and (ii) a subsequent MLP decoder maps this representation to the correct label $h(X) = z$. For X', the same mechanism yields $h(X') = 0$.

Thus GET can separate X and X' on this task, while K-layer MPNNs cannot, completing the proof. □

B.4 Illustrative Example

To make the theoretical advantage more concrete, we consider a specific scenario where nodes A and B are not within k hops in any historical subgraph, nor do their memories interact via a direct chain, but both regularly interact with a third node C (e.g., each Monday). A k-hop GNN's aggregation cannot bridge A and B due to the lack of a sufficiently short path. Similarly, a memory-based model's update chains do not propagate information between A and B, except possibly via C's easily overwritten memory, which struggles to capture long-term periodicity. In contrast, GET's global attention can directly relate any '(A,C)' and '(B,C)' event pairs across the entire history, robustly detecting the periodic co-occurrence regardless of the path.

B.5 Empirical Inference Throughput Analysis

In the standardized TGB evaluation protocol, the dominant cost at test time comes from repeatedly scoring large candidate sets for each query event. All deep temporal models considered in this paper share the same asymptotic per-candidate scoring cost $O(Kd)$ once node representations are available. The observed throughput differences therefore come from how often these representations need to be refreshed and how effectively computation can be batched.

In memory-based architectures such as TGN, each new batch of events triggers recurrent memory updates and, depending on the implementation, repeated neighborhood fetching and message aggregation before candidate scoring. In practice, this leads to substantial overhead on large graphs with long histories.

GET instead performs a single self-attention pass over a sliding window of recent events to obtain a shared contextual representation, and then reuses this representation to score all K candidate destinations in parallel with lightweight MLP heads. Under the TGB workloads we consider, this "encode once, score many" pattern yields the empirical throughput gains reported in Fig. 5, Fig. 6 and Table 16, even though the theoretical $O(Kd)$ scoring term is the same. We therefore view GET's efficiency advantage as an *operational* property under this protocol, not as a fundamentally different asymptotic complexity class.

C Experimental Details

This appendix provides a comprehensive overview of the datasets, baseline models, and implementation details used in our experimental evaluation, ensuring the reproducibility of our work.

C.1 Datasets and Evaluation Metrics

We conduct experiments on two categories of datasets to comprehensively evaluate the proposed framework:

Large-scale datasets: We adopt five publicly available datasets from the Temporal Graph Benchmark (TGB) as shown in Table. 7, namely tgbl-wiki, tgbl-review, tgbl-coin, tgbl-comment, and tgbl-flight. These datasets are widely used in dynamic graph evaluations and present significant challenges such as structural sparsity, node expansion, and continuous-time dynamics. Following the standard 70%/15%/15% train/validation/test split, we report **Mean Reciprocal Rank (MRR)** as the primary evaluation metric.

Small-scale datasets: To further assess generalization capability, we include six publicly available small-scale dynamic graph datasets, namely MOOC, LastFM, Enron, Can. Parl., US Legis., and UN Trade. These datasets, as shown in Table 8 emphasize behavioral dynamics and interaction regularities, making them suitable for evaluating performance under cold-start, weak structural, and

Table 7: Statistics of TGB benchmark datasets. The surprise index [22] (i.e., $|E_{\text{test}} \setminus E_{\text{train}}|/|E_{\text{test}}|$) measures the ratio of test edges that are unseen during training; a lower value indicates that memorization-based methods can potentially achieve strong performance. See TGB documentation for more.

Dataset	tgbl-wiki	tgbl-review	tgbl-coin	tgbl-comment	tgbl-flight
Nodes (users/items)	8,227 / 1,000	352,636 / 298,590	638,486	994,790	18,143
Edges	157,474	4,873,540	22,809,486	44,314,507	67,169,570
Timestamps	152,757	6,865	1,295,720	30,998,030	1,385
Density (%)	1.91×10^{0}	4.63×10^{-3}	5.60×10^{-3}	4.48×10^{-3}	2.04×10^{1}
Bipartite	✓	✓	×	×	×
Surprise	0.108	0.987	0.120	0.823	0.024
Weighted (W)	×	✓	✓	✓	×
Directed (Di)	✓	✓	✓	✓	✓
Attributed (A)	✓	×	×	✓	✓

non-uniform interaction scenarios. We report **Average Precision (AP)** to facilitate fair comparison with prior works.

A detailed description of each dataset's origin and characteristics is provided below, with full statistics available in the official TGB documentation.

C.2 Baseline Descriptions

To ensure a comprehensive and fair evaluation, we compare GET against a suite of twelve representative baselines, which can be broadly categorized into two major methodological paradigms based on their core mechanism for capturing temporal dynamics.

Memory-Based Methods. These models focus on maintaining an explicit, evolving memory state for each node, which is continuously updated by the event stream to capture historical dependencies.

- **JODIE** [15]: Employs coupled Recurrent Neural Networks (RNNs) to learn the embedding trajectories of users and items, capturing their state evolution over time.
- **DyRep** [26]: Models the dynamics of interactions using temporal point processes, focusing on the intensity and association strength between nodes.
- **TGN** [24]: A foundational framework that introduces a dedicated memory module for each node, which is updated via a message-passing scheme upon new interactions.
- **EdgeBank** [22]: A lightweight and efficient non-parametric method that predicts future links based on a cache of recent historical edges between nodes.

Structure-Based Methods. These methods focus on encoding the graph's topological structure at or near the time of prediction. They typically operate on sampled temporal subgraphs, historical interaction patterns, or temporal walks to capture structural context.

- **TGAT** [33]: One of the first models to apply self-attention to dynamic graphs, aggregating features from a node's temporal neighbors with a functional time encoding.
- **CAWN** [30]: Captures the structural evolution of a node's neighborhood by sampling and encoding causal anonymous walks.
- **TCL** [28]: A contrastive learning framework that models causal relationships by constructing cascading interaction graphs.
- **GraphMixer** [4]: A simple yet effective model that uses an MLP-Mixer architecture to process a node's historical interaction sequence, challenging the necessity of complex graph structures.
- **NAT** [20]: Builds a dictionary of multi-hop neighbors for each node to efficiently capture structural information and updates node states via an RNN.
- **PINT** [25]: An expressive model that uses an injective message passing scheme and incorporates relative positional features based on temporal walk counting.
- **DyGFormer** [38]: A powerful Transformer-based model that captures local structure by encoding neighbor co-occurrence as "patches" and processes these patches with a Transformer.
- **TNCN** [42]: A hybrid model that enhances TGN's memory mechanism with features based on multi-hop Neural Common Neighbors, explicitly injecting structural heuristics. We categorize it as structure-based due to its core novelty lying in the structural feature engineering.

To ensure a fair and rigorous comparison, the results for these baseline models were obtained as follows:

- For the **TGB** datasets, all baseline results are directly taken from the official, continuously updated TGB leaderboard to ensure consistency with prior works.
- For the **six small-scale datasets** (MOOC, LastFM, etc.), we reproduced the results by running the experiments ourselves. For most baselines, we utilized the implementations provided in the **DyGLib** library, loading their officially tuned best hyperparameter configurations. For models not included in DyGLib (i.e., NAT and PINT), we used their official public codebases and performed a grid search to find the best hyperparameters.
- For the **inference speed experiments**, results for DyRep, EdgeBank, and TGN were obtained using the official TGB example scripts. Results for all other baselines were generated using the **DyGLib_TGB** repository, loading their optimal configurations to ensure a fair comparison of practical runtime.

C.3 GET Implementation Details

All experiments for our model (GET) were conducted on a server equipped with an NVIDIA GeForce RTX 4090 GPU (24 GB memory) and Intel Xeon Platinum 8370C 128-core CPUs. All experiments

Table 8: Statistics of benchmark dynamic graph datasets. Surprise index as defined in [11]. "Repeat ratio" and "Density" are computed as in [11]. "Bipartite": ✓ indicates a bipartite graph; × indicates otherwise. "Domain" is based on the original dataset description.

Property	MOOC	Enron	Can. Parl.	US Legis.	UN Trade	LastFM
Nodes	7,144	184	734	225	255	1,980
Total Edges	411,749	125,235	74,478	60,396	507,497	1,293,103
Unique Edges	178,443	3,125	51,331	26,423	36,182	154,993
Timestamps	345,600	22,632	14	12	32	1,283,614
Duration	17 months	3 years	14 years	12 congresses	32 years	1 month
Time Granularity	Unix timestamps	Unix timestamps	years	congresses	years	Unix timestamps
Density (%)	60.2	376.0	41.6	119.9	783.5	132.0
Bipartite	✓	×	✓	✓	✓	✓
Domain	Interaction	Social	Politics	Politics	Economics	Interaction
N&L Feat	– & 4	– & –	– & 1	– & 1	– & 1	– & -

Table 9: Key hyperparameters for GET and its variants on TGB datasets.

Category	Hyperparameter	Value
General Parameters	Hidden Dimension (d_{model})	256
	Batch Size	256
	Learning Rate (Initial)	4e-4
	Optimizer	AdamW
	LR Scheduler	CosineAnnealingWarmRestarts
	Loss Weight (λ)	0.4
	Max window length (L_{seq})	dataset-specific (tuned; typically 1k to 4k)
	Eval. step size	256 (large-scale) / 32 (small-scale)
Transformer	Num. Layers	4
	Num. Heads	8
	Feed-Forward Dim.	1024
	Attention Dropout	0.15
	Residual Dropout	0.1
GNN Enhancement	Num. GNN Layers (GAT)	4
	GNN Hidden Dimension	256
	GNN Dropout	0.1
	Num. Decay Rates (k)	8
Memory Enhancement	Memory Dimension	256
	Message Dimension	256

were conducted using PyTorch version 2.1 on a server running Ubuntu 22.04 with CUDA version 12.2. To ensure reproducibility, we provide a detailed list of the key hyperparameters used for training GET and its variants across all TGB datasets. Unless otherwise specified, we use a batch size of 256, a hidden dimension of 256, 4 Transformer layers, and 4 GNN layers (for the GET+GNN variant). The AdamW optimizer is used with an initial learning rate of 4×10^{-4} and a Cosine Annealing with Warm Restarts scheduler. All reported results are averaged over five independent runs.

For small-scale datasets (including tgbl-wiki, MOOC, Enron, Can. Parl., US Legis., UN Trade, and LastFM), the evaluation step size is set to 32 to accommodate the smaller number of edges; for all large-scale TGB datasets, the default evaluation step size is 256.

A complete list of hyperparameters is provided in Table 9. These parameters were determined via grid search on the validation set of each respective dataset.

C.4 Parameter Sensitivity Analysis

To evaluate the robustness of GET with respect to architectural design, we conduct a sensitivity analysis on the tgbl-wiki dataset, varying the number of GNN layers and Transformer layers respectively.

All other hyperparameters are fixed to the values listed in Table 10. When varying GNN layers, the Transformer depth is fixed to 3; when varying Transformer layers, the GNN depth is fixed to 4.

Table 10: Fixed hyperparameters used in the sensitivity analysis.

Hyperparameter	Value
Batch size	1280
Weight decay	5×10^{-5}
Event dropout	0.0
Graph size	1024
Step size	32
Time loss weight	0.3

The results, shown in Figure 7(a) and Figure 7(b), reveal a clear "sweet spot" for both parameters. We observe a significant performance gain from 2 to 3 GNN layers (MRR 0.5625 $\rightarrow$ 0.7400), underscoring the importance of sufficient structural depth. A similar pattern appears with Transformer layers, peaking at 4 layers (MRR 0.7477) and showing robustness from 2–5 layers. These trends suggest that GET is robust across a range of model depths and does not rely on precise hyperparameter tuning.

C.5 Full-Official Window Sensitivity on tgbl-review

The tgbl-review stream is extremely sparse, and our best-performing configuration uses the memory-based initializer. To complement the inference-time truncation study on tgbl-wiki (Section 6.4), we run a *full-official* window sweep for **GET+Memory** on tgbl-review (full test split and official negatives). Following standard practice for memory models, we reset the memory state and warm up on the released train+validation history before evaluating the test split, scoring each query edge before applying its update. Table 11 shows that larger windows can substantially improve accuracy in this regime.

Table 11: Full-official window length sensitivity on tgbl-review for GET+Memory (single checkpoint).

Model	L_{seq} (tokens)	MRR	Hits@10	Throughput (q/s)
GET+Memory	256	0.1411	0.3770	6649.1
GET+Memory	512	0.1612	0.3832	12438.9
GET+Memory	1024	0.2284	0.4327	17216.6
GET+Memory	2048	0.3935	0.5772	17003.2

C.6 Full-Official Attention Controls on tgbl-review

To probe whether performance on tgbl-review relies on broad self-attention, we repeat the same evaluation with modified attention masks while keeping the checkpoint fixed. Table 12 shows that restricting attention to a causal local band (span=256) results in similar MRR, while disabling attention produces a noticeable drop.

Table 12: Full-official inference-time attention controls on tgbl-review for GET+Memory (single checkpoint, L_{seq}=2048).

Variant	MRR	Hits@10	Throughput (q/s)
GET+Memory (full)	0.3935	0.5772	17003.2
GET+Memory (local, span=256)	0.3921	0.5752	17184.5
GET+Memory (no-attn)	0.3783	0.5574	18319.8

C.7 Robustness to GNN Encoder Choice

To assess whether GET is sensitive to the specific choice of structural encoder, we evaluated GCN and GraphSAGE variants on the test set.

Table 13: Comparison of GNN encoder variants in GET+GNN (Test MRR).

Encoder	tgbl-wiki	tgbl-flight
GCN	0.738 ±0.005	0.882 ±0.004
GraphSAGE	0.741 ±0.006	0.889 ±0.003
GAT (default)	**0.752 ±0.012**	**0.895 ±0.003**

As shown in Table 13, while GAT achieves the best performance, other variants remain competitive, confirming that GET is not strictly tied to a specific GNN architecture.

C.8 Impact of Positional Encoding Schemes

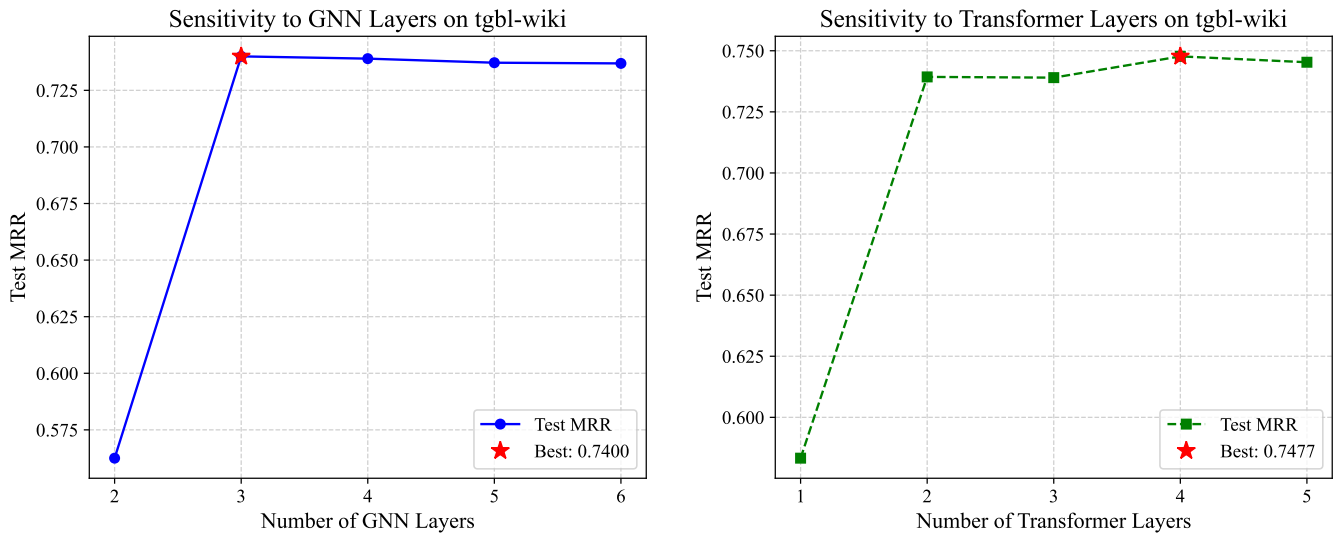
We also compared several positional / temporal encoding schemes within the Transformer backbone of GET, including learned absolute positional embeddings, RoPE-style rotary encodings, and ALiBi. All variants use the same GET backbone and training protocol; only the positional encoding mechanism is changed. Table 14 shows validation MRR on tgbl-wiki. We observe that all three schemes achieve comparable performance, with differences well within the standard deviations of our main experiments. ALiBi performs slightly better on average and, importantly, is parameter-free and robust to changes in sequence length and timestamp range, which motivates our choice to use it as the default in all main results.

Table 14: Effect of different positional encoding schemes on GET (validation MRR on tgbl-wiki and tgbl-flight).

Positional encoding	tgbl-wiki	tgbl-flight
Learned absolute positions	0.728 ±0.053	0.853 ±0.018
RoPE	0.750 ±0.005	0.876 ±0.041
ALiBi (default)	**0.752 ±0.012**	**0.895 ±0.003**

C.9 Sensitivity to the trade-off coefficient λ

We study the sensitivity of GET to the trade-off coefficient λ between the node classification loss and the timestamp regression loss. Table 15 reports validation MRR on three representative datasets when varying $\lambda \in \{0.2, 0.4, 0.6\}$. Across this range, the differences are small and comparable to the reported standard deviations;



(a) Sensitivity to the number of GNN layers.

(b) Sensitivity to the number of Transformer layers.

Figure 7: Sensitivity analysis on the tgbl-wiki dataset. In panel (a), GNN layers are varied while Transformer layers are fixed at 3. In panel (b), Transformer layers are varied while GNN layers are fixed at 4.

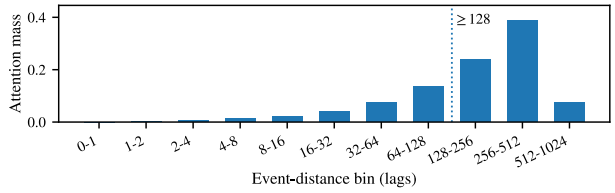


Figure 8: Attention distance profile of GET+Memory on tgbl-review (event-distance bins; self-attention excluded).

$\lambda = 0.4$ consistently offers a slightly better trade-off, which is why we adopt it as the default choice in the main experiments.

Table 15: Sensitivity of GET to the trade-off coefficient λ (Test Metrics).

Dataset	$\lambda = 0.2$	$\lambda = 0.4$ (Default)	$\lambda = 0.6$
tgbl-wiki (MRR)	0.746 ±0.008	**0.752 ±0.012**	0.749 ±0.010
tgbl-comment (MRR)	0.722 ±0.015	**0.732 ±0.017**	0.728 ±0.014
Enron (AP)	94.65 ±0.22	**94.81 ±0.17**	94.70 ±0.19

D Additional Comparisons

D.1 Inference Efficiency and Memory Usage

Efficiency Measurement Protocol. All inference efficiency experiments were conducted on a server equipped with an Intel Xeon Platinum 8370C CPU and NVIDIA RTX 4090 GPUs. To assess practical throughput, we report the end-to-end wall-clock time required for a complete evaluation pass on each dataset. As the inference process for these large-scale benchmarks is deterministic and computationally intensive, a single, uninterrupted run provides a reliable and reproducible measure of real-world performance, which is a standard approach in large-scale systems evaluation. For GPU-accelerated models, measurements were performed with the model as the sole primary task on a dedicated GPU to ensure an isolated and consistent environment. For the CPU-based EdgeBank method, time was measured under similar isolated conditions on the CPU. To maintain a fair comparison of algorithmic efficiency, relative speedup factors (e.g., 21× faster) are calculated exclusively among GPU-accelerated deep learning models.

E Case Study: The tgbl-review Benchmark

The tgbl-review dataset presents an extreme case for dynamic graph learning, as shown in Table 7: it is exceptionally sparse (density 4.63×10^{-3}), with a near-maximal surprise index (0.987), and a minuscule repeat ratio (0.19%). In this "information vacuum," most test edges are one-off, previously unseen interactions, rendering both local structural signals and global sequential patterns exceptionally weak.

This unique scenario exposes the inductive biases and limitations of different model families:

(1) Structure-agnostic models excel when there is little structure to exploit. The best-performing model, GraphMixer (MRR ~0.52), leverages a simple MLP-based design. By not relying on neighborhood aggregation, it avoids overfitting to noisy or non-existent structures, learning a robust mapping from shallow histories.

(2) Memory modules remain valuable in sparse settings. TGN (MRR ~0.38) benefits from its node-level memory, which can aggregate the few available interactions of each entity over time.

Table 16: Inference time (total wall-clock time in H:MM:SS format) on large-scale TGB datasets with diverse scalability challenges. GET is the only strong baseline that runs robustly across all scenarios.

Model	tgbl-wiki	tgbl-review	tgbl-coin	tgbl-flight
EdgeBank*	0:09:01	1:41:06	32:16:27	245:49:35
TGN	0:11:47	0:58:30	5:43:51	1:40:56
DyRep	0:15:42	0:58:26	5:36:10	1:41:26
DyGFormer	1:26:39	4:50:15	7:46:44	OOM/OOT†
TCL	0:10:43	0:28:38	OOM/OOT	OOM/OOT
GraphMixer	0:43:25	1:31:08	OOM/OOT	OOM/OOT
TGAT	1:42:52	5:08:58	OOM/OOT	OOM/OOT
CAWN	4:28:49	8:47:02	OOM/OOT	OOM/OOT
GET+GNN	**0:00:33**	**0:02:59**	**0:25:28**	**1:03:39**

* Run on CPU; time is wall-clock time.
† OOM/OOT (Out of Memory/Out of Time) indicates that the run failed due to resource constraints. NA indicates that the data was not provided for this model.

This mechanism remains effective even when the graph structure is fragmented or the node is isolated.

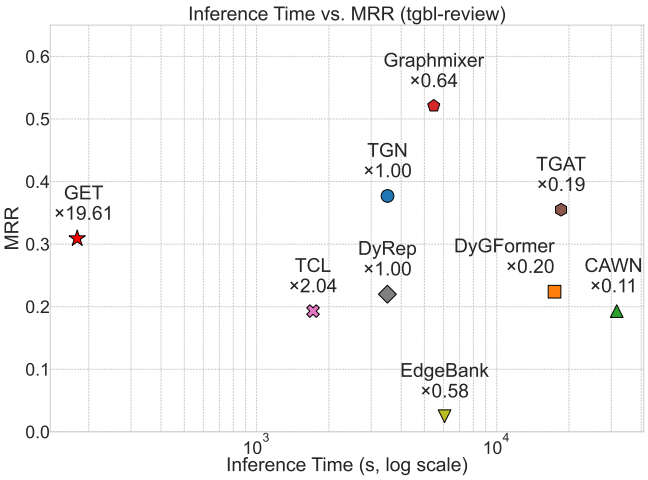


Figure 9: Inference Time vs. MRR on tgbl-review. GET matches or exceeds the accuracy of strong baselines while delivering orders-of-magnitude faster inference.

(3) GET balances competitive performance with strong efficiency. Our GET model achieves a respectable MRR (∼0.31), while delivering up to **19.6× faster inference** compared to memory-based models like TGN (Figure 9). The speedup mainly comes from reusing a single encoded event window when scoring the candidate destinations (*encode once, score many*), instead of performing neighborhood extraction / memory updates for each candidate. Under the TGB protocol, GET still computes scores over the candidate set; the benefit is reduced *repeated encoding* rather than avoiding candidate scoring. While the advantage is smaller on tgbl-review—where histories are short and interactions are sparse—GET remains competitive and offers a favorable accuracy–efficiency trade-off.

(4) Limitations of structural enhancement. Our ablation study shows that GNN enhancements bring little benefit in tgbl-review: with high sparsity and novelty, neighborhood aggregation cannot provide useful predictive information and may even propagate noise. Conversely, incorporating a memory module (GET+Memory) substantially boosts performance, highlighting the promise of hybrid global-local models for such challenging regimes.

The tgbl-review benchmark vividly demonstrates that (i) strong structural assumptions may hurt in sparse, surprise-rich graphs, (ii) entity memory is essential, and (iii) GET offers an attractive compromise between generalization and efficiency. These findings motivate future research on more adaptive, data-aware structural modules.

F Supplementary: Exploratory Results on Synthetic Graph Generation

Scope note. This section is *supplementary* and not part of the paper's main claims. We include it to probe whether the same event-sequence formulation can be adapted to a simple autoregressive baseline for synthetic graph generation. The main paper focuses on discriminative temporal link prediction.

The main body of this paper focuses on discriminative dynamic link prediction. In this appendix, we provide preliminary experiments that probe whether the event sequence model learned by GET can also be used to generate short-horizon temporal interaction sequences on small and medium-sized graphs.

Our goal here is not to introduce a full-fledged temporal graph generator, but rather to examine whether the autoregressive decoder produces structurally and temporally coherent rollouts when applied in a simple generation protocol. To this end, we follow the evaluation setup of TIGGER [9], comparing GET to specialized temporal graph generation baselines such as DYMOND, TagGen, and TIGGER on standard benchmarks.

Table 17: Comparison of GET (adapted as a sequence baseline) against multiple baselines on synthetic dynamic graph generation. Each dataset block reports statistics of the original graph (first row, labeled Median), followed by deviations for generated graphs by DYMOND, TagGen, TIGGER, and an adapted GET baseline (last row per dataset). Lower values indicate better performance.

Dataset	$\lvert\mathcal{V}\rvert$	$\lvert\mathcal{E}\rvert$	T	Method	Time(s)	% Edge overlap	Mean degree	Triangle count	PLE	NC	Global CF
Wiki-Small	1.6K	2.9K	50	Median	-	-	1.1064	0.0	16.4626	44.0	**0.0**
				DYMOND	69120	**0.0**	0.2424	**0.0**	11.426	32.0	**0.0**
				TagGen	1800	87.169	0.0674	**0.0**	5.6519	**0.0**	**0.0**
				TIGGER	14	1.2821	**0.0352**	**0.0**	**4.5005**	4.0	**0.0**
				GET	**4.01**	0.03	0.8101	**0.0**	-	15.0	**0.0**
UC Irvine	1.8K	33K	194	Median	-	-	1.5714	0.0	4.634	14.0	0.0
				TagGen	12, 480	79.356	0.1806	**0.0**	0.7732	**1.0**	**0.0**
				TIGGER	**125**	25.0	**0.076**	**0.0**	**0.4135**	3.0	**0.0**
				GET	768.2	**0.01**	0.2634	**0.0**	2.1994	45.0	**0.0**
Bitcoin	3.7K	24K	191	Median	-	-	1.8443	2.0	3.5607	13.0	0.0102
				TagGen	18579	80.0	0.2311	**0.0**	0.5207	**1.0**	0.0016
				TIGGER	**128**	24.294	**0.1217**	1.0	**0.294**	3.0	0.0078
				GET	776.27	**0.00**	1.0276	3.0	5.3285	523.0	**0.0001**

F.1 Experimental Setup

We adopt the experimental setup from TIGGER [9], where each model is trained on the entire temporal interaction graph and tasked with generating a synthetic graph of identical size (matching the number of nodes and interactions in the original dataset). We evaluate GET against the following generative baselines:

- **Median**: Computes the median values of graph statistics from snapshots of the original graph, serving as a statistical reference (ground-truth baseline) for evaluating the fidelity of generated graphs.
- **DYMOND** [40]: A graphlet-based generative model with cubic computational complexity.
- **TagGen** [43]: An optimization-driven approach employing heuristic-based path sampling.
- **TIGGER** [9]: A state-of-the-art neural generative model employing autoregressive interaction sampling.

Evaluations are conducted on three diverse real-world datasets, including messaging interactions (**UC Irvine** [16]), financial transactions (**Bitcoin** [14]), online discussions (**Reddit** [17]), and collaborative editing patterns **Wiki-Small** [17]). Dataset details are provided in Table 17. Model performance is evaluated across three primary aspects:

- **Efficiency**: Runtime required to generate synthetic graphs.
- **Novelty**: Percentage of edge overlap with the original graph (lower is better).
- **Fidelity**: Median absolute errors across timestamps for essential graph statistics:
  - Mean degree (**Mean Degree**)
  - Triangle count (**Triangle Count**)
  - Power-law exponent (**PLE**)
  - Number of components (**NC**)

Lower median absolute error indicates better fidelity to the original graph characteristics.

F.2 Results and Analysis

Table 17 summarizes the comparative performance of GET and baseline methods. Key observations are summarized as follows.

Regarding **Efficiency**, while TIGGER is faster on these smaller datasets, GET's runtime remains practical and scales better than non-neural baselines like DYMOND and TagGen.

For **Novelty (Edge Overlap)**, GET excels by maintaining near-zero overlap, indicating it generates genuinely new interactions rather than memorizing training data.

Finally, concerning **Graph Fidelity**, GET shows a trade-off. While it struggles to match TIGGER on global statistics like PLE and NC, it accurately models local structures like triangle counts. This suggests GET prioritizes generating novel events with correct local patterns, highlighting a different generative strength compared to baselines.

Overall, these results suggest that GET can serve as a useful *building block* for temporal graph generation, balancing novelty and fidelity while remaining reasonably efficient on the settings we consider. We emphasize that these experiments are preliminary and short-horizon in nature; a more systematic study of long-horizon generation, alternative decoding schemes, and downstream generative applications is an interesting direction for future work.